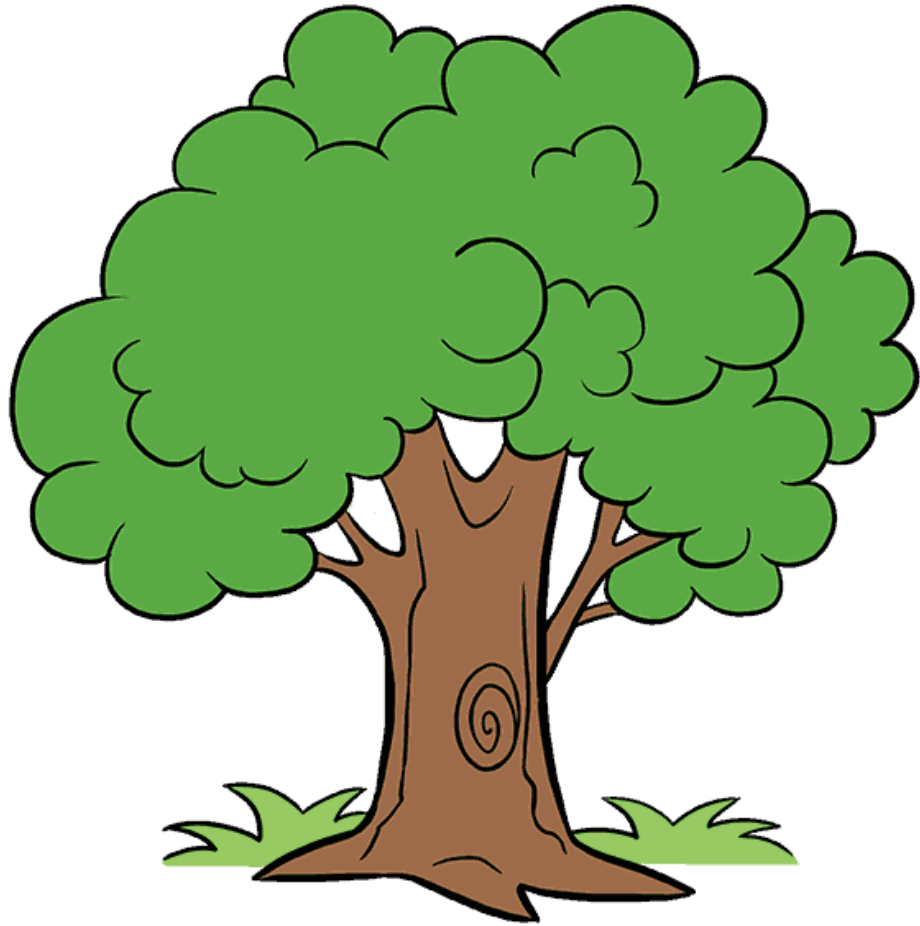
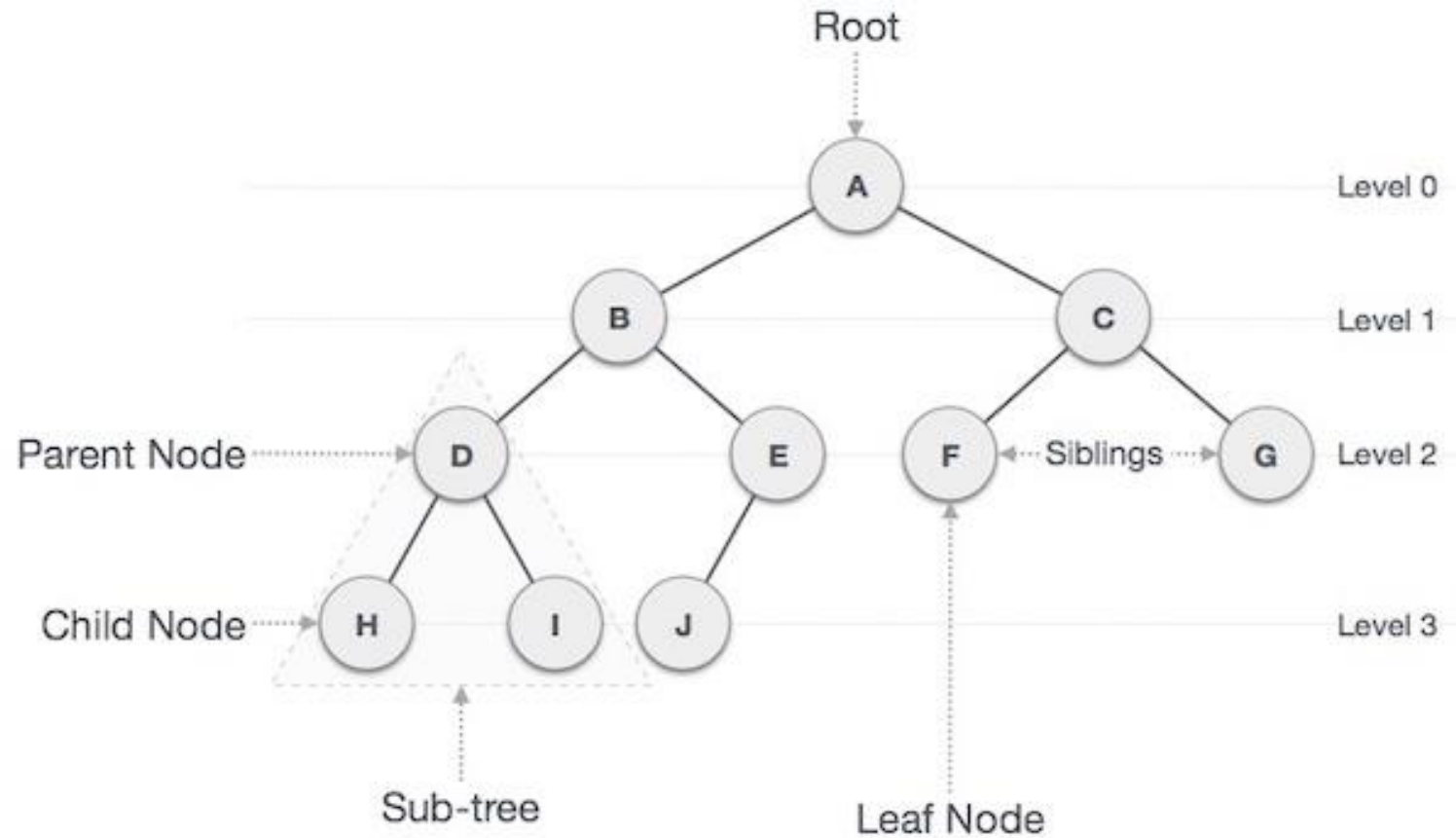




# Trees

## Binary Tree/Binary Search Tree

# Tree terminologies



# Tree Applications

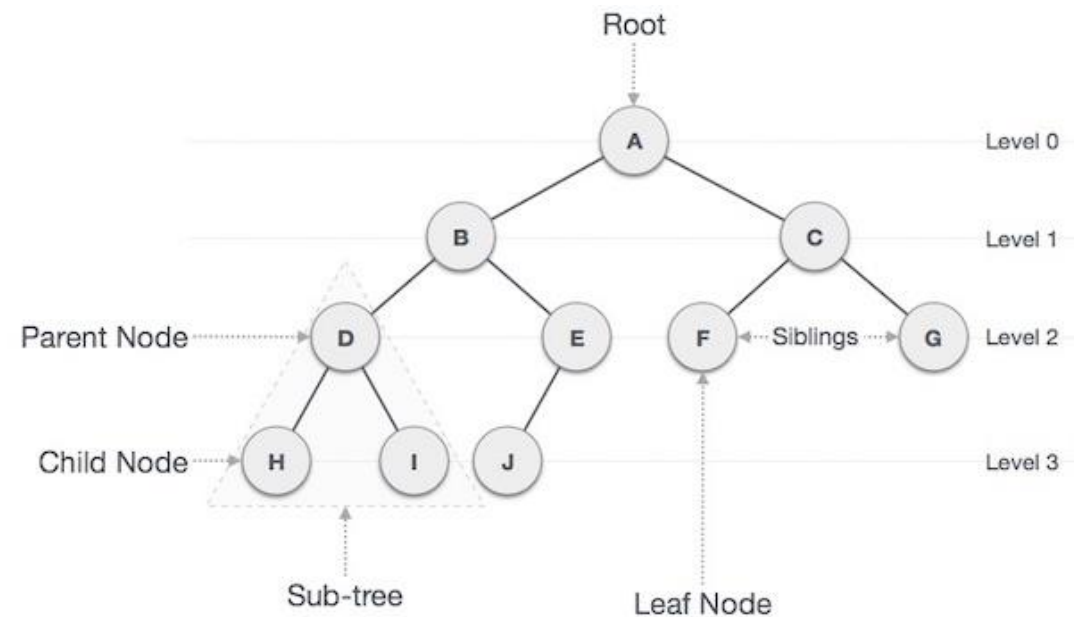
- Organogram
- Family Tree
- Product catalog
- Geographical tree

## Computer Applications

- File System
- Markup text
- Decision tree
- Pdf format

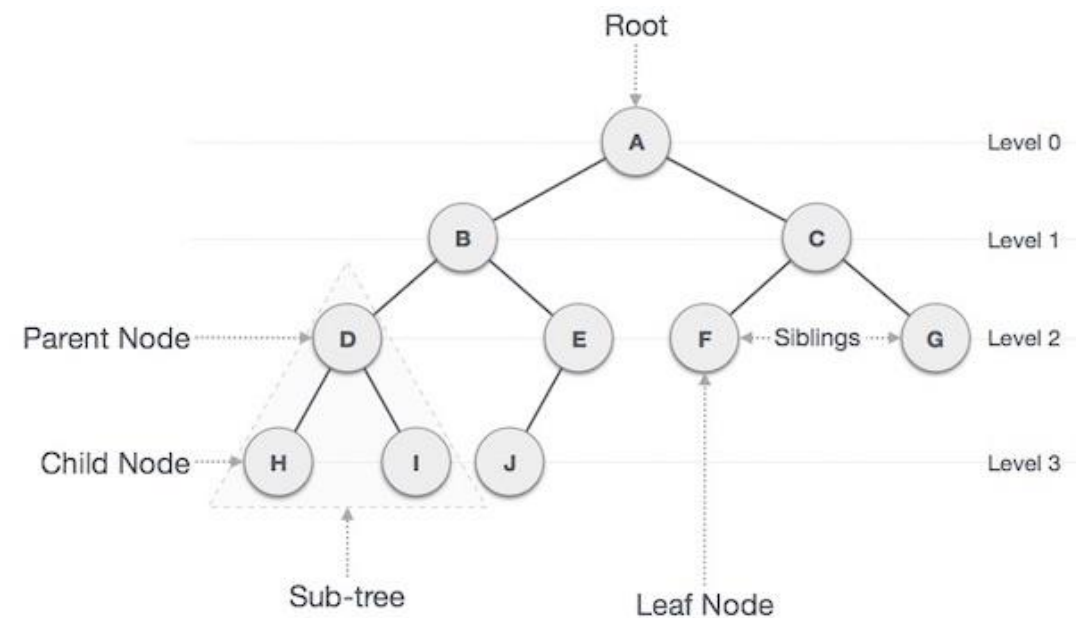
# Tree

- A tree is a collection of nodes connected by directed (or undirected) edges. It has following general properties:
  - One node is distinguished as a **root**;
  - Every node (exclude a root) is connected by a directed edge *from* exactly one other node; A direction is: *parent* -> *children*



# Some terminologies

- Each node can have *arbitrary* number of children.
- Nodes with no children are called **leaves**, or **external** nodes. In the given picture, H, I, J, E, G are leaves.
- Nodes, which are not leaves, are called **internal** nodes. Internal nodes have at least one child.
- Nodes with the same parent are called **siblings**. In the picture and C are siblings, so are D and E.
- The **depth of a node** is the number of edges from the root to the node. The depth of F is 2.
- The **height of a node** is the number of edges from the node to the deepest leaf. The height of B is 2.
- The **height of a tree** is a height of a root.

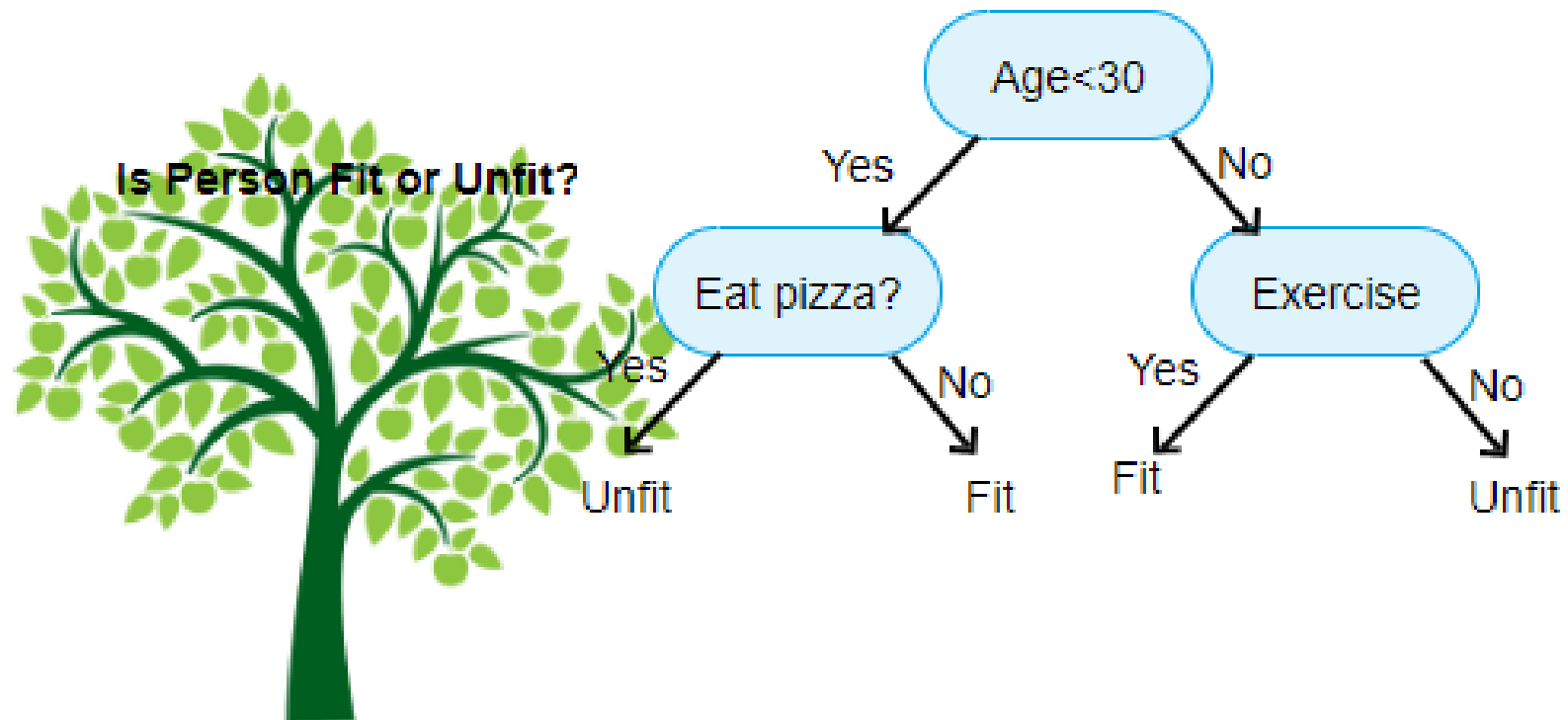


# Some terminologies

- A ***path*** of  $T$  is a sequence of nodes such that any two consecutive nodes in the sequence form an edge.
- A tree is ***ordered*** if there is a meaningful linear order among the children of each node; that is, we purposefully identify the children of a node as being the first, second, third, and so on. Such an order is usually visualized by arranging siblings left to right, according to their order.

# Binary Tree

- A **binary tree** is an ordered tree with the following properties:
  1. Every node has at most two children.
  2. Each child node is labeled as being either a **left child** or a **right child**.
  3. A left child precedes a right child in the order of children of a node.

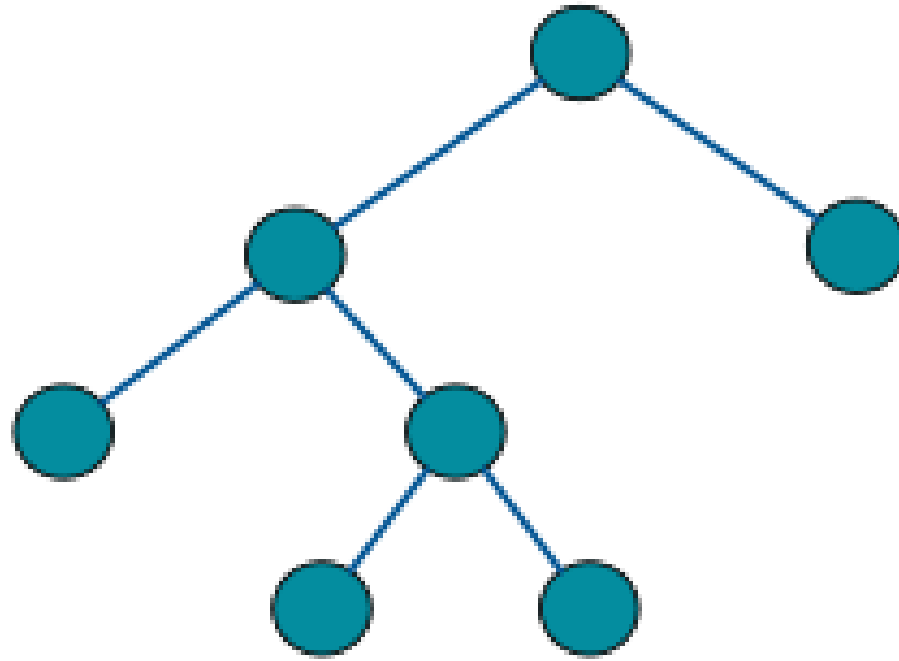


<https://www.aitimejournal.com/@akshay.chavan/a-comprehensive-guide-to-decision-tree-learning>



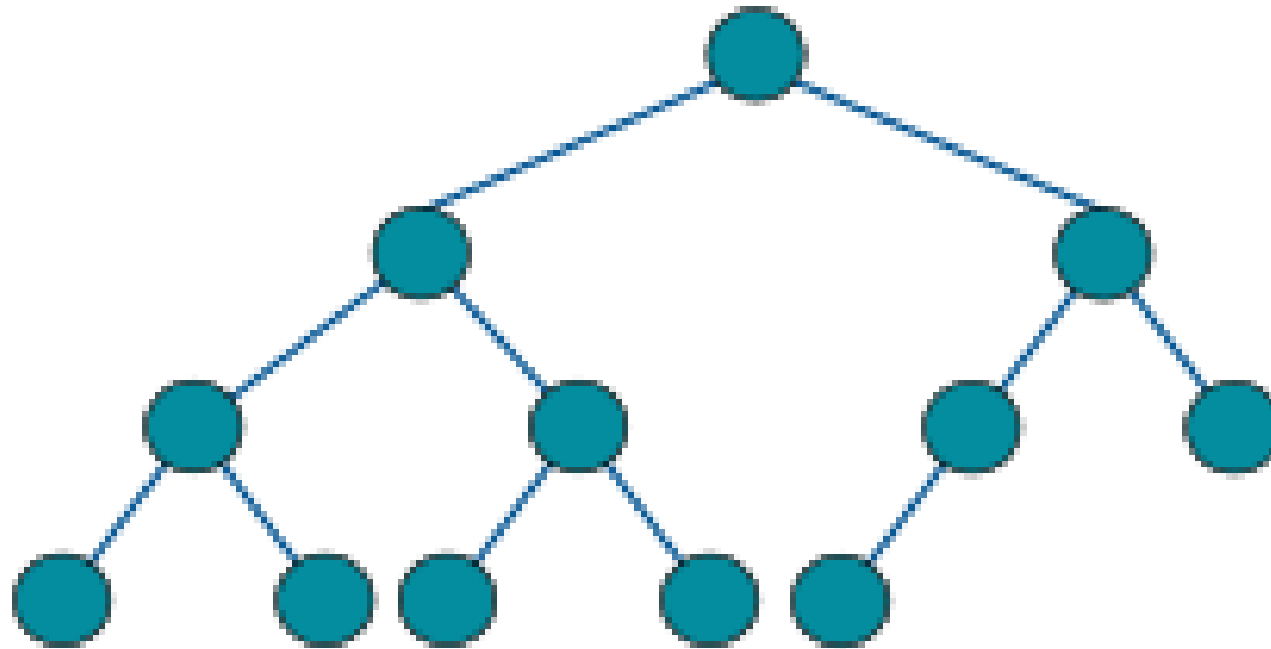
# Full Binary Tree

- A Binary Tree is full if every node has 0 or 2 children



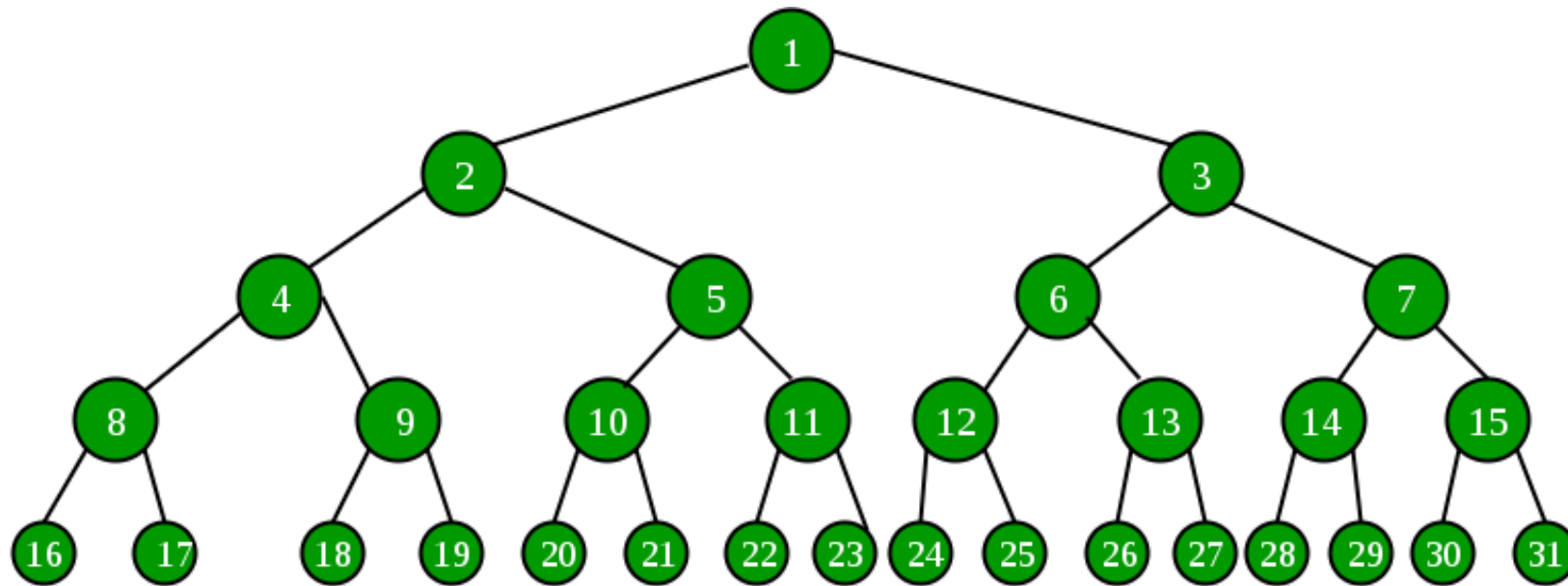
# Complete Binary Tree

- A Binary Tree is complete Binary Tree if all levels are completely filled except possibly the last level and the last level has all keys as left as possible.



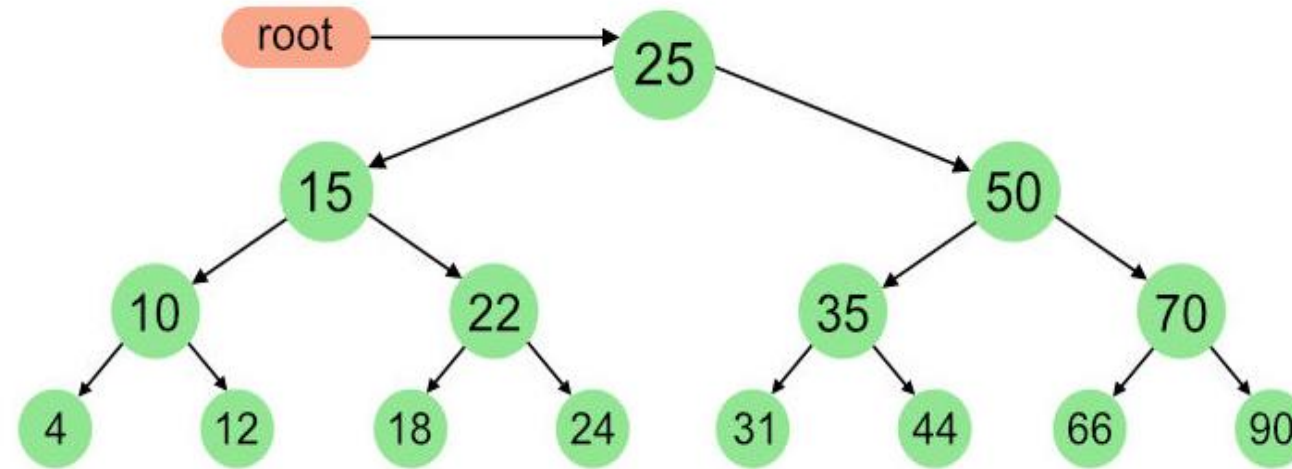
# Perfect Binary Tree

- A Binary tree is Perfect Binary Tree in which all internal nodes have two children and all leaves are at same level.



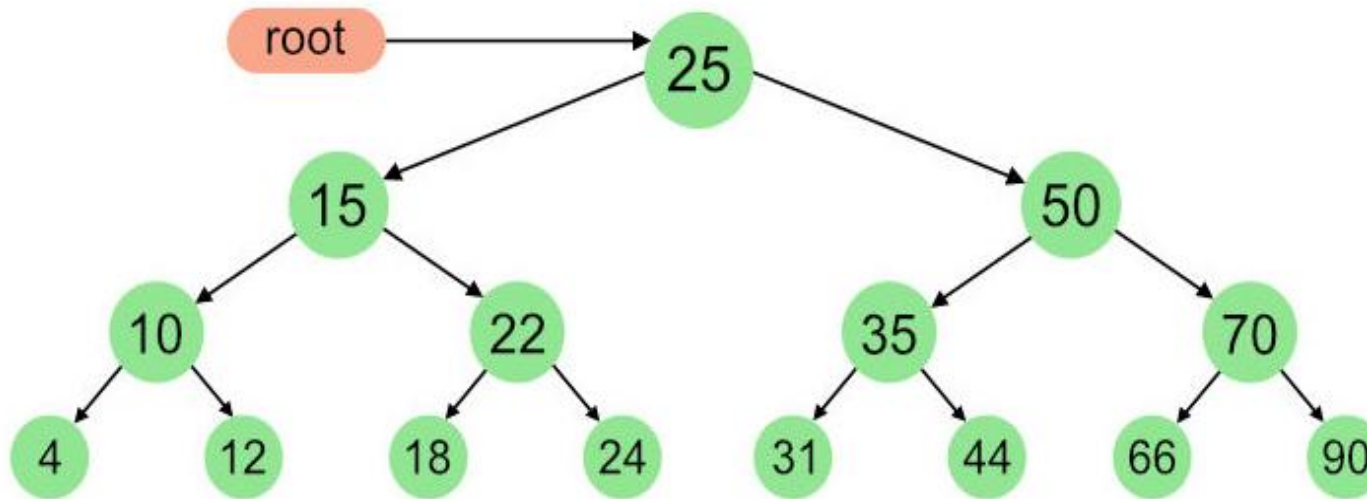
# Tree Traversal

# Tree traversal



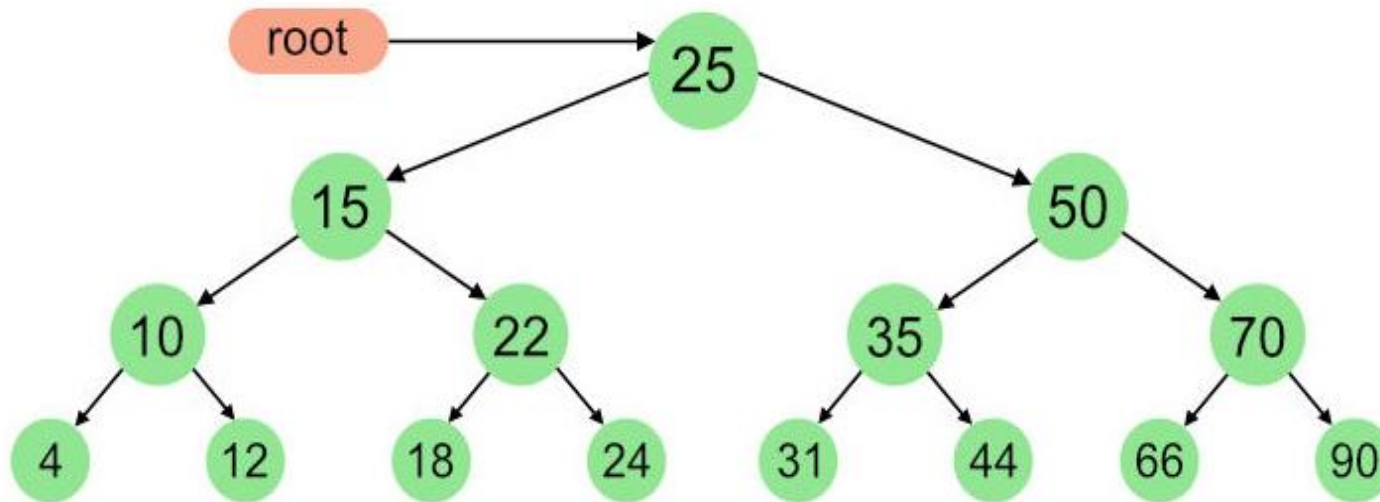
- Run DFS on this tree.

# Preorder traversal



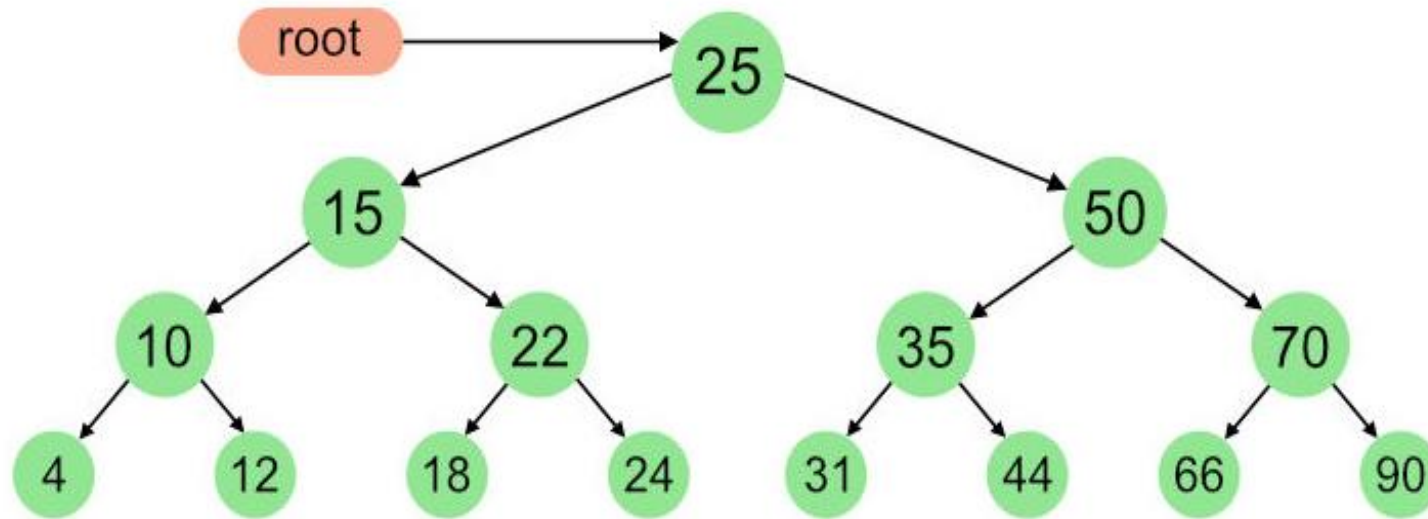
- Pre-order follows this order: parent, left subtree, right subtree
- A pre-order on the given tree will visit nodes in the following order:  
25,15,10,4,12,22,18,24,50,35,31,44,70,66,90

# Inorder traversal



- Inorder follows this order: left subtree, parent, right subtree
- A pre-order on the given tree will visit nodes in the following order:  
4,10,12,15,18,22,24,25,31,35,44,50,66,70,90

# Post-order traversal



- Post-order follows this order: left subtree, right subtree, parent
- A post-order on the given tree will visit nodes in the following order:  
4,12,10,18,24,22,15,31,44,35,66,90,70,50,25



# Preorder, Postorder and Inorder Pseudo Code

## **Algorithm** *Preorder*( $x$ )

**Input:**  $x$  is the root of a subtree.

1. **if**  $x \neq \text{NULL}$
2.     **then** output key( $x$ );
3.         *Preorder*(left( $x$ ));
4.         *Preorder*(right( $x$ ));

## **Algorithm** *Postorder*( $x$ )

**Input:**  $x$  is the root of a subtree.

1. **if**  $x \neq \text{NULL}$
2.     **then** *Postorder*(left( $x$ ));
3.         *Postorder*(right( $x$ ));
4.         output key( $x$ );

## **Algorithm** *Inorder*( $x$ )

**Input:**  $x$  is the root of a subtree.

1. **if**  $x \neq \text{NULL}$
2.     **then** *Inorder*(left( $x$ ));
3.         output key( $x$ );
4.         *Inorder*(right( $x$ ));

<https://www.slideshare.net/ERPunitJain/binary-search-tree-47288612>

# Inorder Traversal

```
def inorder(root):  
    if root is not null:  
        inorder(root.left)  
        print(root.val)  
        inorder(root.right)
```

```
def preorder(root):  
    if root is not null :  
        print(root.val)  
        preorder(root.left)  
        preorder(root.right)
```

```
def postorder(root):  
    if root is not null :  
        preorder(root.left)  
        preorder(root.right)  
        print(root.val)
```

# Using different types of traversal

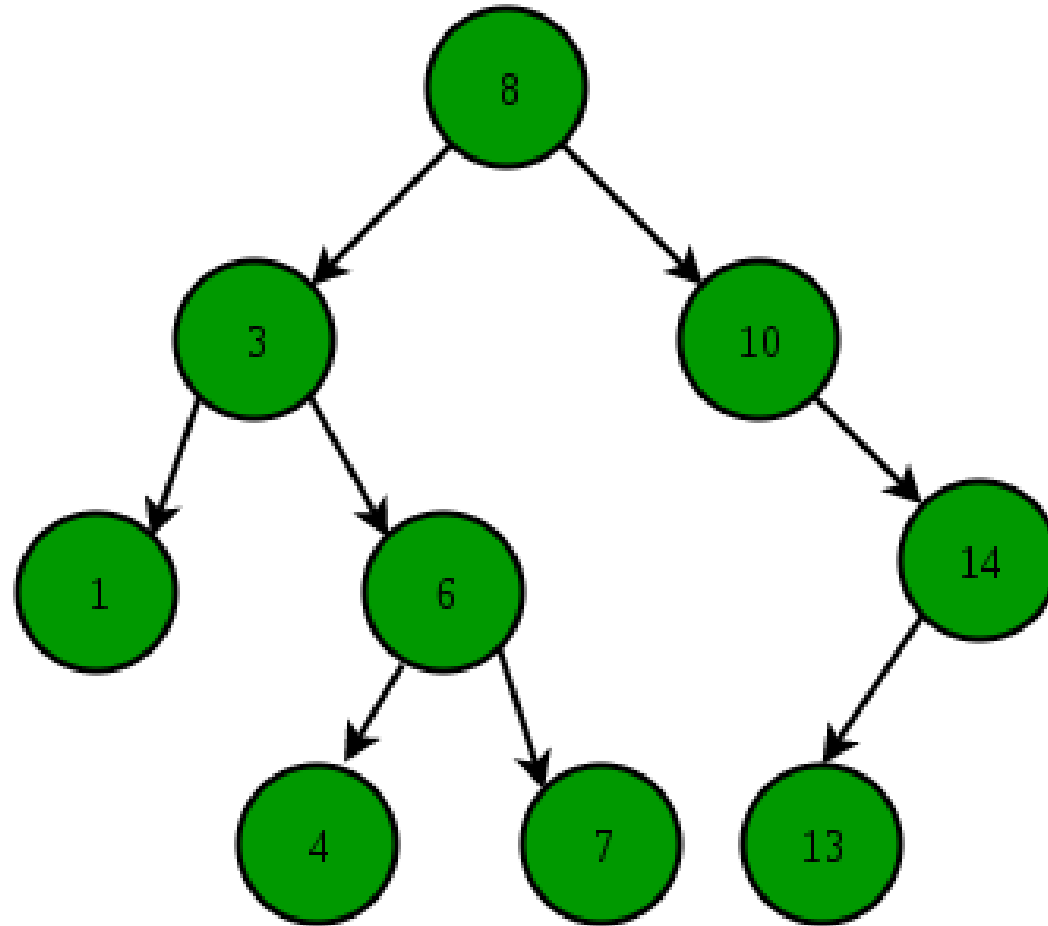
- If you know you need to explore the roots before inspecting any leaves, you pick **pre-order** because you will encounter all the roots before all of the leaves.
- If you know you need to explore all the leaves before any nodes, you select **post-order** because you don't waste any time inspecting roots in search for leaves.
- If you know that the tree has an inherent sequence in the nodes, and you want to flatten the tree back into its original sequence, then an **in-order** traversal should be used. The tree would be flattened in the same way it was created.
- A pre-order or post-order traversal might not unwind the tree back into the sequence which was used to create it.

# Tree Traversal - Visualisation

- [http://ceadserv1.nku.edu/longa/classes/mat385\\_resources/docs/traversal.htm](http://ceadserv1.nku.edu/longa/classes/mat385_resources/docs/traversal.htm)

# Binary Search Tree

# Binary Search Tree

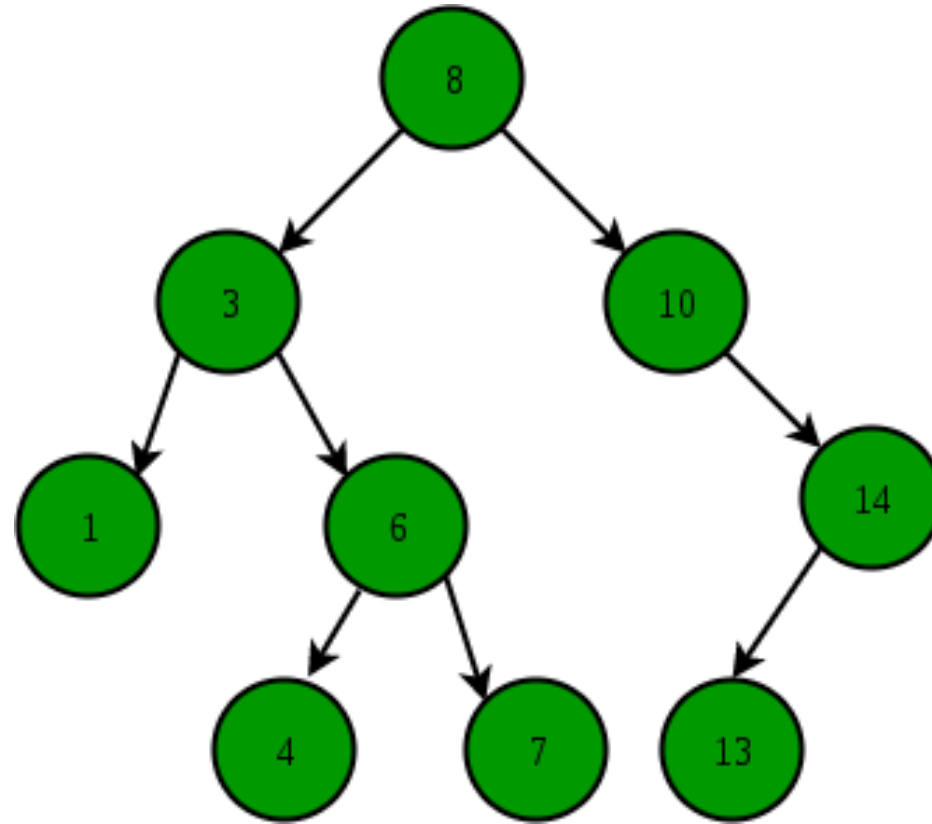


5

# Applications of Binary Tree/Binary Search Tree

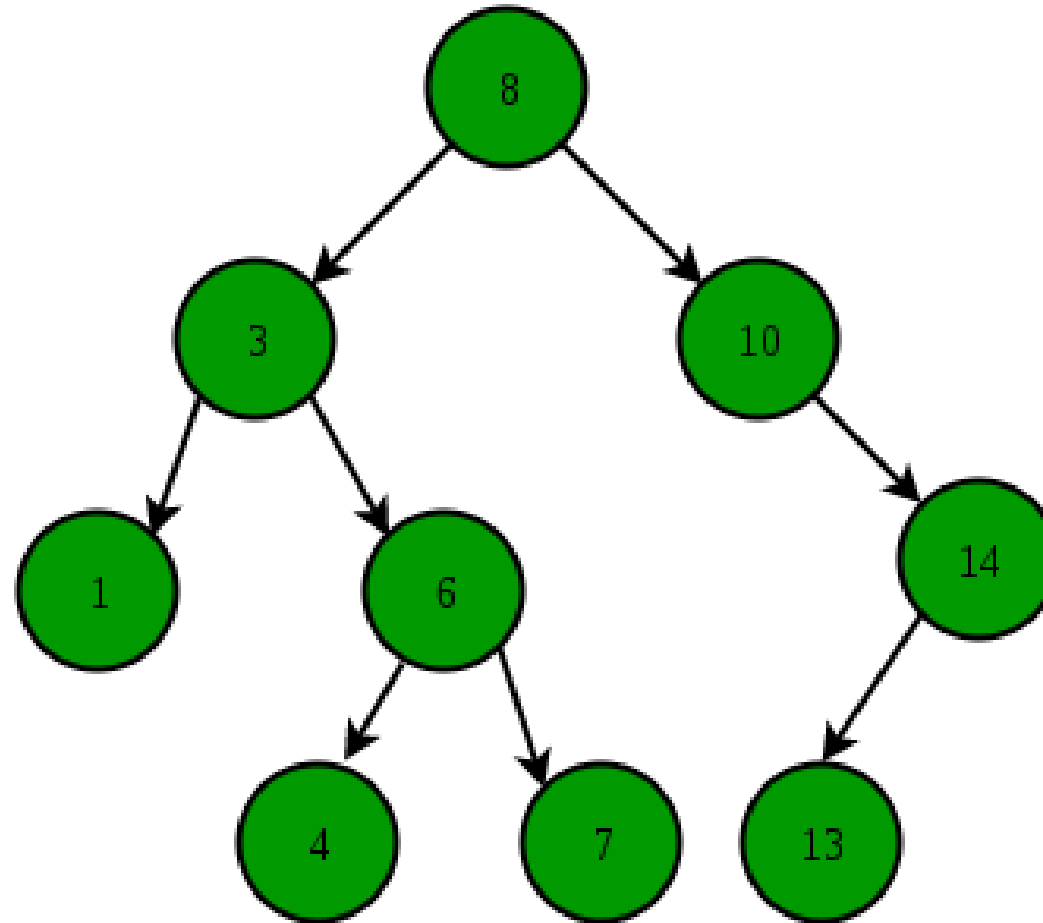
- To solve database problem such as indexing.
- Implementing routing table in router.
- Data compression code
- Implementation of Expression parsers and expression solvers

# How to find a node in a BST?



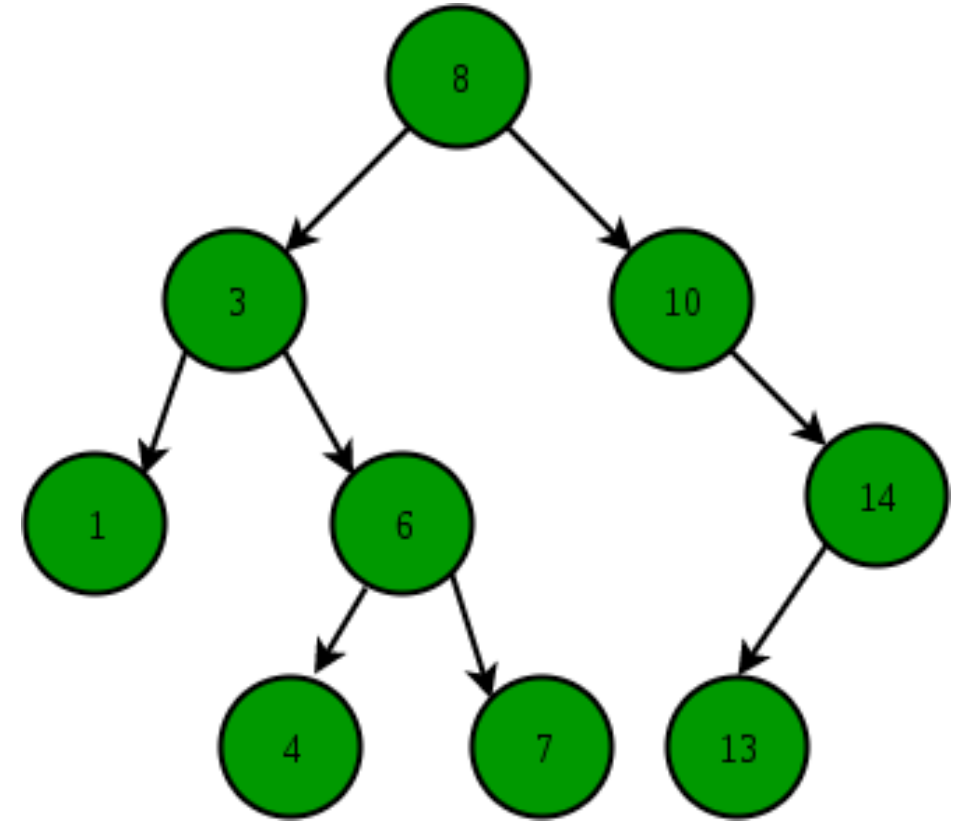


# How to find min or max value in a tree?



# Insertion in BST

- Start from root.
- Compare the inserting element with root, if less than root, then recurse for left, else recurse for right.
- After reaching end, just insert that node at left(if less than current) else right.
- [Binary Search Tree Visualization](#)



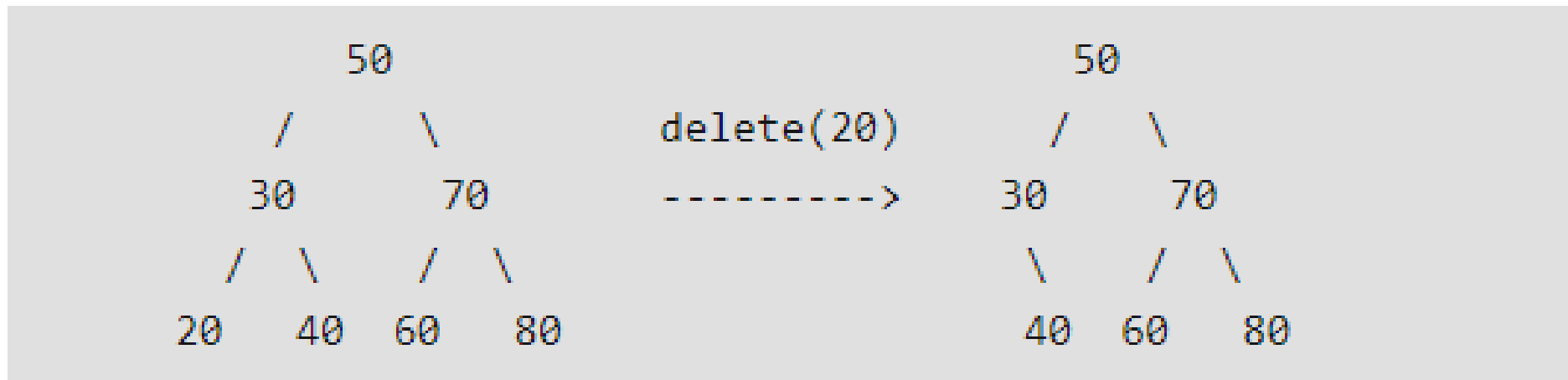
# Binary Search Tree Visualizer

<https://visualgo.net/bn/bst>

<http://btv.melezinek.cz/binary-search-tree.html>

# Deletion in BST – Case 1

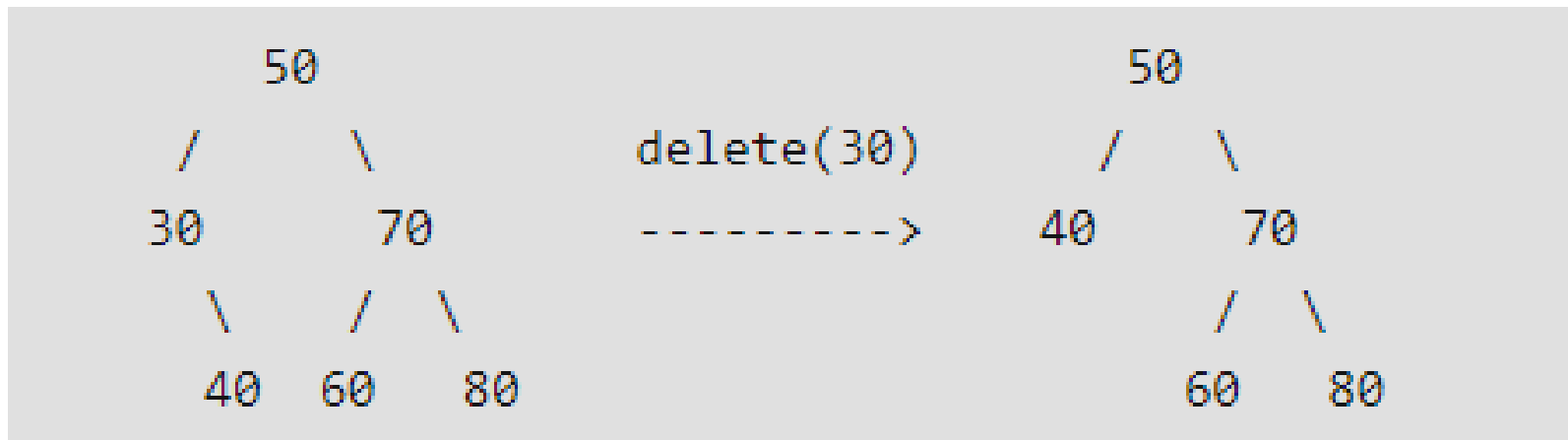
- Node to be deleted in a leaf: Simply remove from the tree



<https://www.geeksforgeeks.org/binary-search-tree-set-2-delete/>

# Deletion in BST – Case 2

- ***Node to be deleted has only one child:***

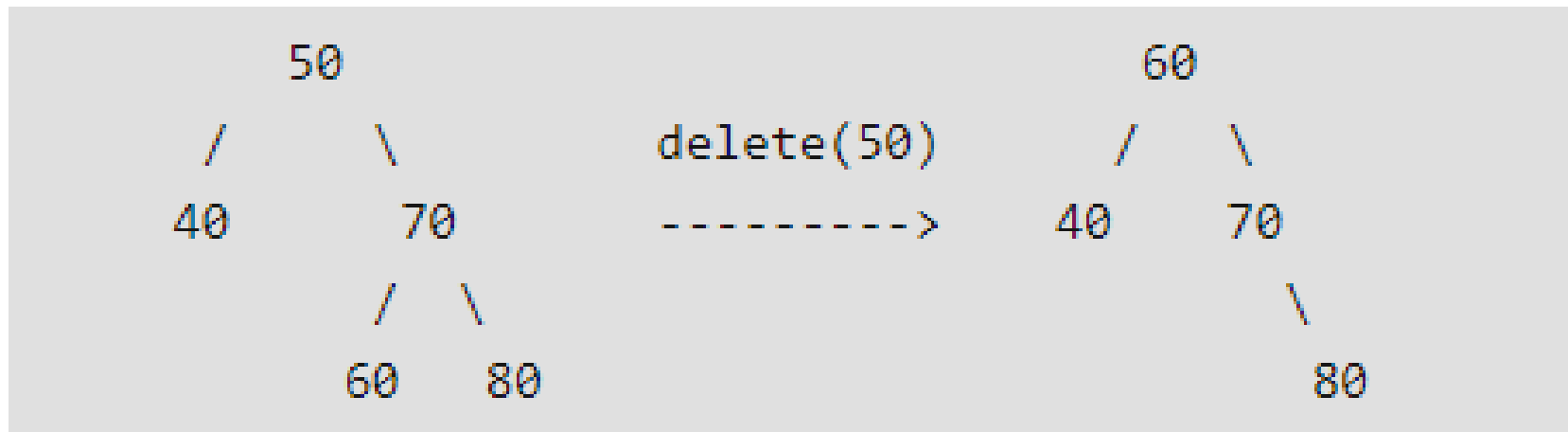


- Copy the child to the node and delete the child

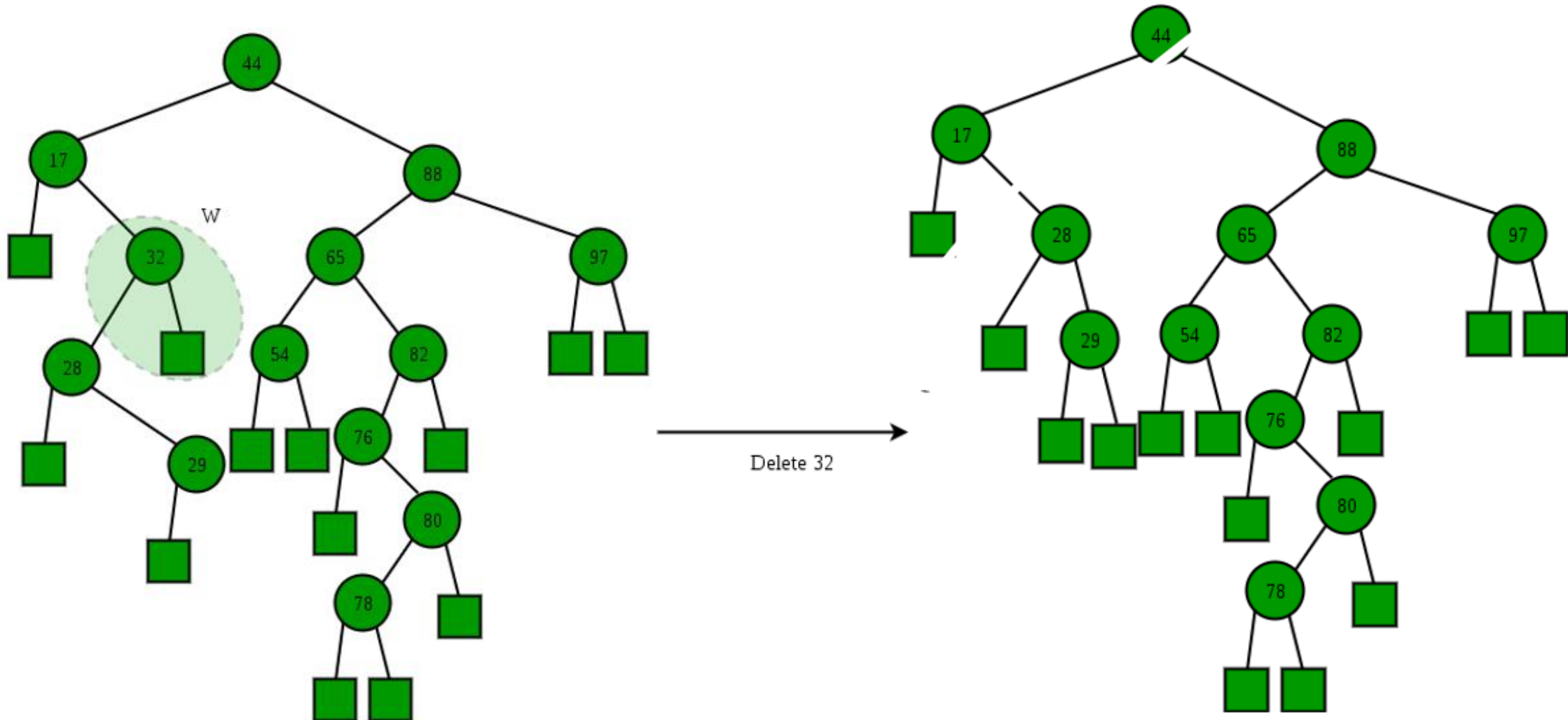
<https://www.geeksforgeeks.org/binary-search-tree-set-2-delete/>

# Deletion in BST – Case 3

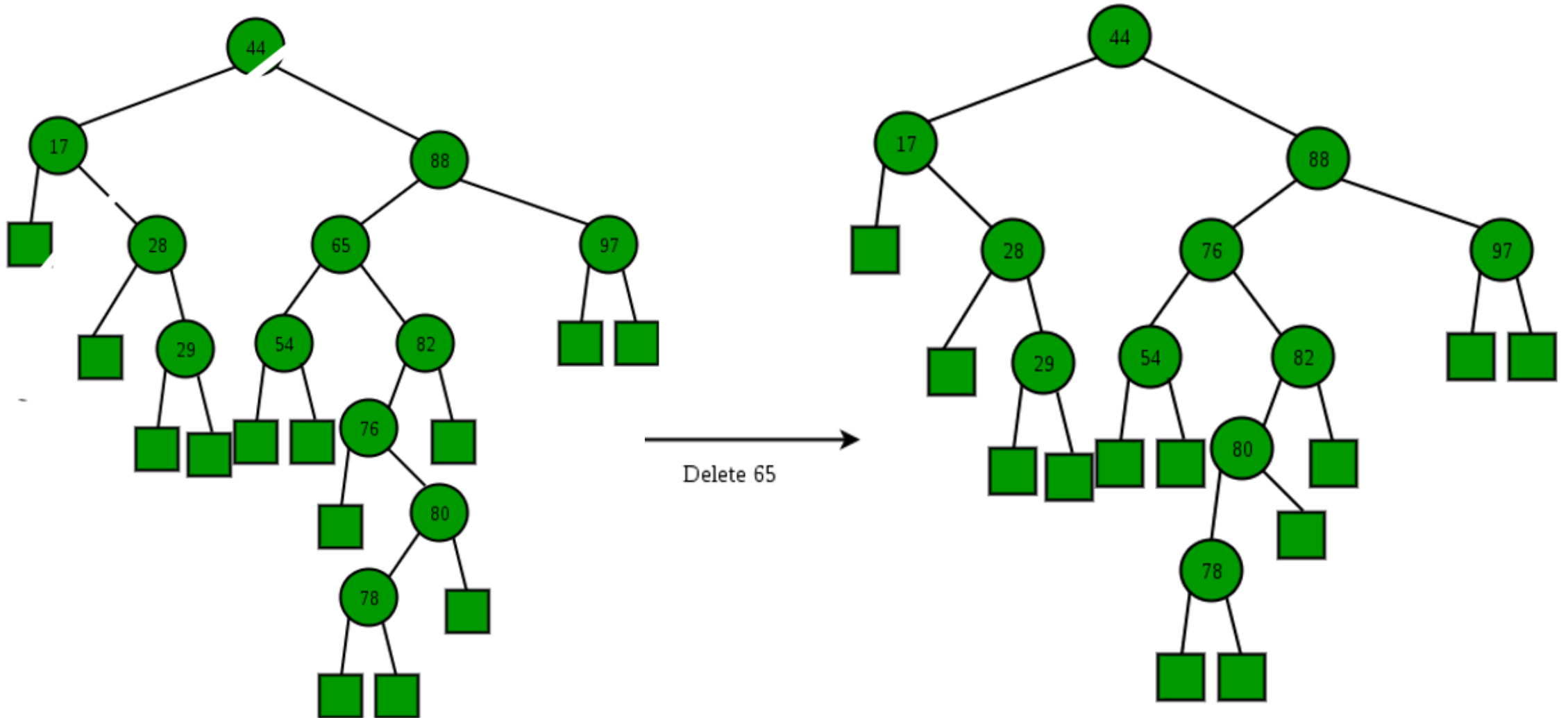
- ***Node to be deleted has two children:***



1. Find *inorder successor* of the node.
  2. Copy contents of the inorder successor to the node
  3. delete the inorder successor.
- Note that *inorder predecessor* can also be used.



<https://www.geeksforgeeks.org/binary-search-tree-set-2-delete/>



<https://www.geeksforgeeks.org/binary-search-tree-set-2-delete/>



# Binary Search Tree – Find/Insert/Delete Visualization

# Thanks